Technical Report: Student Performance Analytics API

Github repo link:
https://github.com/Heortappel-hub/Web-API
API Document:
https://github.com/Heortappel-hub/Web-API/blob/main/api_Document%20(1).pdf
Slides Link(Onedrive):
Student Performance.pptx

Overview:

This report documents the design, development, and deployment of a RESTful API for managing and analysing student performance data. The API enables users to store student information, import data via CSV files, and analyse how various factors influence exam performance.

Architecture Choice:
Architecture pattern
The project follows the Model-View-Template (MVT) architecture pattern inherent to Django, adapted for API development.
Files and purposes:
- Models: Define data structure
- Views: Handle api logic and request processing
- Serializers: Transform data between Python objects and JSON

RESTful Design:
- Resource-based URLs (/api/students/, /api/batches/)
- HTTP methods indicate actions (GET, POST, PUT, PATCH, DELETE)
- Stateless communication with token-based authentication
- Consistent JSON response format

Technology stack:

Programming Language: Python

Reasons: 1. Strong ecosystem for web development and data analysis 2. Wide community support and comprehensive documentation.

Framework: Django + Django REST Framework

Reasons: To implement a RESTful API design, Django REST Framework (DRF) was chosen due to its seamless integration with Django, convenient development workflow, and built-in browsable API interface that allows endpoints to be tested without requiring a frontend application.

Database: SQLite

Reasons: SQLite was chosen as the database solution for this project because it requires zero configuration and integrates seamlessly with Django. Since SQLite stores data in a single file, deployment becomes simpler and more portable. However, SQLite also has certain limitations.

Because it is a file-based database, it does not perform well under high levels of concurrent write operations. For larger production systems with higher traffic, a more scalable database system such as PostgreSQL would be recommended.

Deployment: PythonAnywhere

Reasons: PythonAnywhere offers built-in support for Django projects and allows applications to be deployed using a easy configuration(WSGI). And it also provides a free tier for a small scale project.

Tests and challenges:

The API was tested by browser with Django REST Framework's Browsable API from submissions and response formats.

Register

User Registration - POST /api/register/

Creates a new user account and returns an authentication token.

```
GET /api/register/

HTTP 405 Method Not Allowed
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "detail": "Method \"GET\" not allowed."
}
```

- HTML form
- Raw data

Username

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address

Password

POST

Media type:

application/json

Content:

```
{
    "username": "",
    "email": "",
    "password": ""
}
```

POST

It also used CLI to input test commands such as:

- Registration:

curl -X POST /api/register/ -d '{"username":"test","password":"pass123456"}'

- CRUD operations:

curl -H "Authorization: Token xxx" /api/students/

curl -X POST -H "Authorization: Token xxx" /api/students/ -d '{"exam_score":85}'

- Analysis:

curl -H "Authorization: Token xxx" /api/batches/1/analysis/

The main challenges was taken place on implementations of correlation analysis and deployment. The correlation analysis required implementing the statistical algorithm. In this case, the pearson correlation was chosen to calculate the relations between the hours studied, attendance, sleep hours, tutoring sessions, physical activity and students' exam grades.

$r = \Sigma[(x_i - \bar{x})(y_i - \bar{y})] / \sqrt{[\Sigma(x_i - \bar{x})^2 \times \Sigma(y_i - \bar{y})^2]}$

During the deployment process, several difficulties were encountered while configuring the virtual environment. Many dependencies failed to install properly, which caused runtime errors when running the application. This issue was resolved after generating and using a requirements.txt file to manage the required packages. Another challenge occurred when configuring the WSGI file. Due to unfamiliarity with the deployment setup, the system initially failed to locate the correct project path because the automatically generated path differed from the environment configured in the Bash console. After further investigation and consulting documentation, the configuration was corrected and the application was successfully deployed.

Limitations and potential improvements:

Limitations:

SQLite Database
- Single-file database limits concurrent write operations
- Not suitable for high-traffic production environments

Basic Authentication
- Token-based auth lacks expiration mechanism
- No refresh token support

No Automated Tests
- Manual testing only; no unit test suite implemented.

Potential Future Improvements:

Database Migration
- Migrate to PostgreSQL for production scalability

Enhanced Authentication
- Implement JWT with token expiration

Feature Additions
- Add user roles
- Implement data export (Excel, PDF reports)

Add automated tests

Generative AI Declaration and Analysis:

Generative AI tools were used during the development of this project, mainly through GitHub Copilot (Claude-based assistant). The AI assistant was used to support several stages of the development process, including data processing, data import, documentation formatting, algorithm exploration, and debugging.

During the data preparation stage, AI was used to help analyse the structure of the dataset and assist in writing scripts to process and import CSV data into the database. It helped generate initial code for parsing CSV files and suggested ways to improve the data import process when handling larger datasets. These suggestions were reviewed and adjusted based on the actual requirements of the project.

AI was also helpful when organising and formatting project documentation. It provided guidance on structuring the README file, API documentation, and parts of the technical report, helping ensure that the documentation was clear and logically organised.

In addition, the AI assistant was used to search for and understand suitable algorithms for analysing relationships within the dataset. For example, it helped identify appropriate methods for measuring statistical relationships between variables and provided explanations that supported the implementation.

During development and deployment, AI was frequently used to help analyse errors and debugging issues. When runtime errors occurred, the assistant helped identify possible causes and suggest solutions.

Overall, the use of Generative AI improved development efficiency and helped speed up problem solving. At the same time, all generated content was carefully reviewed to ensure correctness and understanding before being included in the final implementation.